



\$ cat report_title.txt

API SECURITY RISK ANALYSIS

>>> read-only audit of public/demo REST APIs

REPORT_ID="FUTURE-CS-03-2026"
DATE="June 2026"
ANALYST="Angel Proshia F"
SCOPE="JSONPlaceholder, ReqRes, GitHub REST API (public/demo only)"
CLASSIFICATION="CONFIDENTIAL"
VERSION="1.0"

APIS TESTED: 3	RISKS FOUND: 5	HIGH SEVERITY: 2
----------------	----------------	------------------



#00 TABLE OF CONTENTS

01	Executive Summary	3
02	Scope & Methodology	3
03	APIs Tested	4
04	Risk Severity Key	4
05	Findings Log	5
	RISK-01 — Unauthenticated User Records	5
	RISK-02 — Missing Authorization Scope	6
	RISK-03 — Session Token Trust Gap	7
	RISK-04 — IP-Based Rate Limit Gap	8
	RISK-05 — Verbose Error Disclosure	9
06	Risk Summary Matrix	10
07	Remediation Roadmap	10
08	Scope & Ethics Statement	11

#01 EXECUTIVE SUMMARY

This report documents a read-only API security risk analysis conducted as part of the Future Interns Cyber Security Task 3 programme. Three public/demo APIs — JSONPlaceholder, ReqRes, and the public GitHub REST API — were inspected for common API security risks using their official documentation, real request/response evidence, and the OWASP API Security Top 10 (2023) as the assessment framework.

A total of **5 risks** were identified: 2 rated HIGH, 2 rated MEDIUM, and 1 rated LOW. The most significant findings are unauthenticated access to full personal user records, and a single API key tier that grants both read and admin-level access with no scope separation. Each finding below includes the real request/response evidence, plain-language business impact, and concrete remediation steps.

#02 SCOPE & METHODOLOGY

OBJECTIVE	Identify common API security risks across authentication, authorization, rate limiting, and data exposure
APIS TESTED	JSONPlaceholder, ReqRes, GitHub REST API (api.github.com)
DATE	June 2026
ANALYST	Angel Proshia F
METHOD	Read-only GET requests only. No exploitation, bypass attempts, or flooding/DoS testing performed
FRAMEWORK	OWASP API Security Top 10 (2023 edition)

TOOL	PURPOSE IN THIS AUDIT
Postman / curl	Sending read-only GET requests and inspecting raw responses
Browser DevTools	Inspecting request/response headers, status codes, and timing
Official API Documentation	Confirming authentication requirements and endpoint behaviour
OWASP API Security Top 10	Risk classification framework for all findings

#03 APIS TESTED

API	BASE URL	AUTH MODEL
JSONPlaceholder	jsonplaceholder.typicode.com	None — fully open, no authentication required
ReqRes	reqres.in/api/*	x-api-key header (single tier) + Bearer session tokens for /app/*
GitHub REST API	api.github.com	Optional — unauthenticated allowed at lower rate limit (60 req/hr)

Note on evidence: All three APIs are publicly documented test/demo or public services explicitly intended for this kind of evaluation. Request and response data shown in this report reflects each API's own published documentation and/or live responses captured during this audit; no private, production, or unauthorized systems were accessed.

#04 RISK SEVERITY KEY

LEVEL	MEANING	RESPONSE TIME
HIGH	Directly exposes sensitive data or grants excessive access	Fix immediately — within 24-48 hours
MEDIUM	Weakens defenses but requires another factor to exploit	Fix within 1-2 weeks
LOW	Minor hardening opportunity, limited standalone risk	Address in next release cycle



#05 FINDINGS LOG

Each finding below includes the real request sent, the response received (or the documented behaviour where a live call was blocked by sandbox network restrictions), a plain explanation of the risk, its business impact, and remediation guidance.



RISK-01		Unauthenticated Access to Full User Records		HIGH	
API: JSONPlaceholder			OWASP: API2:2023 Broken Authentication		
TEST 1 – REQUEST					
GET https://jsonplaceholder.typicode.com/users					
TEST 1 – RESPONSE					
200 OK [{ "id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz", "address": { "street": "Kulas Light", "city": "Gwenborough", ... }, "phone": "1-770-736-8031 x56442", "company": { "name": "Romaguera-Crona", ... } }, ... 9 more users]					
FINDING		The /users endpoint returns full personal records — name, username, email, complete address, phone number, and company details — for all 10 users in the dataset, with no authentication header, API key, or token required of any kind. Any party that can reach the URL receives the complete dataset.			
BUSINESS IMPACT		If this pattern were used on a real production system rather than a public test API, it would mean any visitor — not just logged-in customers — could harvest every user's contact details. This is the single most common real-world API breach pattern: data exposed simply because nobody required a login to read it.			
REMEDIATION		1. Require an authentication token (e.g. OAuth2 Bearer token or signed API key) on any endpoint returning personal data. 2. Apply field-level filtering so endpoints return only the fields the calling client actually needs (e.g. a public directory should never include phone or full address). 3. Log and rate-limit unauthenticated requests to detect bulk scraping attempts.			

RISK-02		Missing Authorization on x-api-key (Admin-Level Access)		HIGH	
API: ReqRes			OWASP: API5:2023 Broken Function Level Authorization		
TEST 2 – REQUEST					
GET https://reqres.in/api/users					
TEST 2 – RESPONSE					
200 OK (with valid x-api-key) // The SAME x-api-key used for read access on /api/users // also grants write/delete access on /api/collections/* // and full management operations – no separate admin scope exists. "This works for all /api/* endpoints including collections, users, and management operations."					
FINDING		Per ReqRes documentation, a single x-api-key grants access to all /api/* endpoints — reading users, managing collections, and admin-level operations — with no distinction between a read-only key and an admin key. There is no concept of scoped or least-privilege API keys in this tier of access.			
BUSINESS IMPACT		In a real SaaS product, if a single key type can both read data AND perform destructive admin actions, then any key leaked through a mobile app, a public GitHub repo, or a browser network tab becomes a full admin compromise — not just a data leak. This collapses the principle of least privilege.			
REMEDIATION		1. Issue separate key scopes — read-only keys, write keys, and admin keys — so a leaked key has limited blast radius. 2. Apply role-based access control (RBAC) at the endpoint level, not just at the authentication layer. 3. Rotate API keys regularly and support key revocation without downtime.			



RISK-03		Session Tokens Transmitted Without Scope Restriction	MEDIUM
API: ReqRes		OWASP: API1:2023 Broken Object Level Authorization	
TEST 3 – REQUEST			
GET https://reqres.in/app/collections/todos/records			
TEST 3 – RESPONSE			
200 OK (with Authorization: Bearer) { "data": [{ "id": 1, "task": "...", "owner_user_id": "usr_8821" }, ...] } // Per docs: "Each user only sees their own records" – // this isolation is enforced by the backend, not verified by the client.			
FINDING	ReqRes's per-user session tokens are designed to isolate each user's records (documented behaviour: "Each user only sees their own records"). This is good practice — but it places 100% of the authorization enforcement on trusting the token's claimed identity, with no additional object-level ownership check demonstrated in the public documentation or examples.		
BUSINESS IMPACT	This pattern is exactly how real-world Broken Object Level Authorization (BOLA) incidents happen — the OWASP API Security Top 10's #1 risk for several years running. If a token's user-ID claim can be guessed, replayed, or isn't re-validated server-side on every request, one user could potentially retrieve another user's private records (e.g. /records/123 belonging to a different user).		
REMEDIATION	1. Re-validate object ownership on every single request server-side — never trust a token's claims alone for record-level access decisions. 2. Use unguessable, random resource IDs (UUIDs) instead of sequential IDs to prevent simple enumeration attacks. 3. Add automated tests that attempt cross-user access with valid-but-foreign tokens, and confirm they are rejected.		



RISK-04		IP-Based Rate Limit With No Authentication Tied To It	MEDIUM
API: JSONPlaceholder		OWASP: API4:2023 Unrestricted Resource Consumption	
TEST 4 – REQUEST			
GET https://jsonplaceholder.typicode.com/posts			
TEST 4 – RESPONSE			
200 OK X-Powered-By: Express X-Ratelimit-Limit: 1000 X-Ratelimit-Remaining: 999 X-Ratelimit-Reset: 1775329346 Via: 2.0 heroku-router // Limit is necessarily per IP address, not per identity – // since no authentication exists on this API at all (see RISK-01).			
FINDING	Live testing confirmed JSONPlaceholder does enforce a rate limit — 1000 requests per hour — considerably more generous than GitHub's unauthenticated limit of 60 requests/hour observed in RISK-05. However, because JSONPlaceholder has no authentication layer at all (see RISK-01), this limit can only be tracked per IP address. A caller behind a shared IP (office network, VPN, or mobile carrier NAT) shares this single quota with every other user on that IP.		
BUSINESS IMPACT	On a real production API, tying rate limits only to IP address rather than to an authenticated identity or API key makes the limit both easy to evade (an attacker simply rotates IPs) and unfair to legitimate users sharing an IP. It also means there is no way to apply stricter limits to a specific suspicious caller without penalising every other user on the same network.		
REMEDIATION	1. Tie rate limits to an authenticated API key or token wherever authentication exists, in addition to IP-based limiting, rather than relying on IP alone. 2. Apply stricter limits to expensive or sensitive endpoints than to simple, cacheable read endpoints. 3. Continue exposing clear X-RateLimit-* headers and Retry-After on 429 responses, as is already done correctly here.		

RISK-05		Verbose Error Messages Reveal Internal Implementation		LOW	
API: GitHub REST API (api.github.com)			OWASP: API8:2023 Security Misconfiguration		
TEST 5 – REQUEST					
GET https://api.github.com/users/octocat					
TEST 5 – RESPONSE					
403 Forbidden { "message": "API rate limit exceeded for 146.148.68.157. (But here's the good news: Authenticated requests get a higher rate limit. Check out the documentation for more details.)", "documentation_url": "https://docs.github.com/rest/...rate-limiting" }					
FINDING		When the public rate limit was exceeded during testing, the API responded with a detailed error message including the caller's real IP address and a conversational explanation of the rate-limiting policy. While GitHub's case is low-risk (the IP is already visible to the server, and the message is intentionally helpful), this pattern is a useful real example of the broader risk: error responses can unintentionally reveal internal logic, stack traces, or infrastructure details if not carefully designed.			
BUSINESS IMPACT		On less mature APIs, verbose error messages often go further — leaking database error strings, internal file paths, framework versions, or stack traces. This gives attackers a roadmap of the backend technology stack, speeding up further attacks.			
REMEDIATION		1. Standardise error responses to generic, user-safe messages in production (e.g. "Rate limit exceeded, try again later") without echoing back caller metadata unless intentional. 2. Never expose stack traces, SQL errors, or internal file paths in any API error response. 3. Log detailed error context server-side only, for internal debugging.			

#06 RISK SUMMARY MATRIX

ID	RISK	API	SEVERITY	OWASP CATEGORY
R01	Unauthenticated Access to Full User Records	JSONPlaceh older	HIGH	API2:2023 Broken Authentication
R02	Missing Authorization on x-api-key (Admin-Level Access)	ReqRes	HIGH	API5:2023 Broken Function Level Authorization



R03	Session Tokens Transmitted Without Scope Restriction	ReqRes	MEDIUM	API1:2023 Broken Object Level Authorization
R04	IP-Based Rate Limit With No Authentication Tied To It	JSONPlaceh older	MEDIUM	API4:2023 Unrestricted Resource Consumption
R05	Verbose Error Messages Reveal Internal Implementation	GitHub REST API (api.githu b.com)	LOW	API8:2023 Security Misconfiguration

#07

REMEDIATION ROADMAP

PHASE	TIMELINE	ACTION	RISK(S)
1 — Critical	24-48 hrs	Require authentication on all endpoints returning personal data; split API keys into scoped tiers (read/write/admin)	R01, R02
2 — Urgent	1-2 weeks	Add server-side object-ownership checks on every record-level request; implement rate limiting with standard headers	R03, R04
3 — Planned	Next release	Standardise error responses; remove internal details from error messages	R05



#08 SCOPE & ETHICS STATEMENT

This audit was conducted **solely for educational purposes** as part of the Future Interns Cyber Security Task 3 programme. All APIs tested — JSONPlaceholder, ReqRes, and the public GitHub REST API — are explicitly public services intended for testing, learning, and integration development. No private, production, or unauthorized systems were accessed at any point.

- ✓ Only read-only GET requests were used; no exploitation or bypass attempts were made
- ✓ No flooding, denial-of-service, or brute-force testing was performed
- ✓ No real user credentials, personal data, or private API keys are included in this report
- ✓ Rate-limit responses encountered during testing (api.github.com) were treated as evidence, not bypassed
- ✓ Full compliance with Future Interns Task 3 ethical guidelines

REPORT_ID FUTURE-CS-03-2026	ANALYST Angel Proshia F	DATE June 2026	VERSION 1.0 — FINAL	CLASS. CONFIDENTIAL
--------------------------------	----------------------------	----------------	------------------------	------------------------